

Гайдбук: как защищать токены и секреты в Jupyter Notebook и Google Colab

1. Зачем вообще защищать токены

В аналитике данных и Python-ноутбуках часто используются внешние сервисы: API, базы данных, облачные хранилища, BI-платформы, LLM-сервисы, маркетинговые кабинеты, GitHub, Kaggle и другие системы.

Для подключения к таким сервисам могут использоваться секреты:

- API-ключи;
- токены доступа;
- пароли;
- строки подключения к базам данных;
- OAuth-токены;
- service account JSON;
- cookies;
- SSH-ключи;
- `.env`-файлы;
- файлы вида `kaggle.json`;
- ключи облачных сервисов.

Главное правило:

секрет нельзя писать прямо в коде ноутбука.

Плохой пример:

```
API_KEY = "sk-xxxxxxxxxxxxxxxxxxxxxxxxxx"
```

Такой код опасен, потому что ноутбук можно случайно:

- отправить преподавателю;
- загрузить в LMS;
- залить на GitHub;
- переслать в чат;
- показать на демонстрации экрана;
- сохранить вместе с выводами ячеек;
- передать другому слушателю.

Если токен попал в чужие руки, другой человек потенциально может использовать сервис от вашего имени: делать запросы, тратить лимиты, получать доступ к данным или выполнять действия в системе.

2. Где секрет может случайно утечь в ноутбуке

В ноутбуках секрет может оказаться не только в коде.

Типовые места утечки:

Место	Пример риска
Ячейка кода	<code>API_KEY = "..."</code>
Вывод ячейки	<code>print(API_KEY)</code>
Ошибка выполнения	токен попал в traceback или лог
Markdown-ячейка	ключ вставлен в текст инструкции
История Git	секрет удалили из файла, но он остался в старом коммите
Скриншот	токен виден на демонстрации экрана
Файл <code>.env</code>	файл случайно загрузили в GitHub
Файл конфигурации	<code>kaggle.json</code> , <code>credentials.json</code> , <code>service_account.json</code>
Логи	библиотека вывела URL или заголовки запроса
Общий доступ к Colab	другой пользователь получил доступ к ноутбуку

Важная мысль:

если секрет хотя бы один раз был опубликован, его нужно считать скомпрометированным.

Недостаточно просто удалить строку из ноутбука. Токен мог остаться в истории, кэше, старой версии файла, выводе ячейки или у других пользователей.

3. Базовое правило для учебных ноутбуков

Для учебных работ лучше использовать такую политику:

1. В ноутбуке не должно быть настоящих токенов.
2. В примерах использовать заглушки: `YOUR_API_KEY_HERE`.
3. Для реального запуска токен вводится отдельно.
4. Перед отправкой ноутбука нужно очистить выводы.
5. Перед загрузкой в GitHub нужно проверить, что секреты не попали в файлы.
6. Если токен случайно опубликован, его нужно сразу отозвать или перевыпустить.

Безопасный учебный пример:

```
import os

api_key = os.getenv("OPENAI_API_KEY")
```

```
if not api_key:
    raise ValueError("API-ключ не найден. Добавьте его в переменные окружения.")
```

В таком варианте сам ключ не хранится в ноутбуке. Ноутбук только пытается прочесть его из окружения.

4. Способ 1. Вводить секрет вручную через getpass

Самый простой способ для демонстрации или разовой работы — запросить ключ во время выполнения ноутбука.

```
from getpass import getpass

api_key = getpass("Введите API-ключ: ")
```

Плюсы:

- ключ не записан в коде;
- ключ не сохраняется в `.ipynb`;
- удобно для учебной демонстрации;
- не требует настройки `.env` или облачного хранилища секретов.

Минусы:

- ключ нужно вводить каждый раз заново;
- неудобно для регулярных процессов;
- если случайно вывести переменную через `print`, секрет всё равно появится в выводе.

Правильно:

```
from getpass import getpass

api_key = getpass("Введите API-ключ: ")

# используем api_key в запросе,
# но не печатаем его на экран
```

Неправильно:

```
print(api_key)
```

Когда использовать:

- на учебном вебинаре;
- в разовой демонстрации;

- когда не хочется настраивать окружение;
- когда ноутбук будут пересылать другим людям.

5. Способ 2. Использовать переменные окружения

Переменная окружения — это значение, которое хранится вне кода и доступно программе во время выполнения.

В ноутбуке мы пишем:

```
import os

api_key = os.getenv("OPENAI_API_KEY")
```

Ключевой момент: сам токен находится не в ноутбуке, а в окружении.

Пример для macOS / Linux:

```
export OPENAI_API_KEY="ваш_ключ"
```

Пример для Windows PowerShell:

```
$env:OPENAI_API_KEY="ваш_ключ"
```

В Python:

```
import os

api_key = os.getenv("OPENAI_API_KEY")

if api_key is None:
    raise ValueError("Переменная OPENAI_API_KEY не найдена")
```

Плюсы:

- секрет не хранится в коде;
- удобно для локальной разработки;
- подходит для разных сервисов;
- код можно безопасно отправлять без ключа.

Минусы:

- нужно настроить окружение;
- у слушателей могут быть разные операционные системы;
- переменные окружения могут быть видны в настройках процесса;

- нужно следить, чтобы ключ не выводился в лог.

Когда использовать:

- для локальных проектов;
- для ноутбуков, которые запускаются на своём компьютере;
- для задач, где ключ нужен регулярно;
- для кода, который потом может стать скриптом или приложением.

6. Способ 3. Использовать `.env`-файл для локальной разработки

`.env`-файл — это локальный файл с переменными окружения. Его удобно использовать в учебных проектах и локальной разработке.

Пример файла `.env`:

```
OPENAI_API_KEY=ваш_ключ
DATABASE_URL=postgresql://user:password@host:5432/db
```

В Python:

```
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
```

Но есть важное правило:

файл `.env` нельзя загружать в GitHub, LMS или отправлять другим людям.

Добавьте `.env` в `.gitignore`:

```
.env
*.env
```

Хорошая структура проекта:

```
project/
├─ notebooks/
│   └─ practice.ipynb
├─ data/
```

```
├── .env
├── .env.example
├── .gitignore
└── README.md
```

Файл `.env` хранит реальные секреты.

Файл `.env.example` можно отправлять другим людям:

```
OPENAI_API_KEY=your_api_key_here
DATABASE_URL=your_database_url_here
```

Плюсы:

- удобно для локальной разработки;
- легко объяснить начинающим;
- код остаётся чистым;
- можно хранить несколько переменных.

Минусы:

- `.env` легко случайно отправить;
- это не полноценный менеджер секретов;
- не подходит для командной промышленной работы без дополнительных правил;
- секрет всё равно лежит на диске в обычном файле.

Когда использовать:

- для учебных локальных проектов;
- для небольших персональных ноутбуков;
- для демонстрации правильной структуры проекта;
- когда важно не вставлять ключ прямо в код.

7. Способ 4. Использовать Colab Secrets

Если работа идёт в Google Colab, лучше использовать встроенный механизм Secrets.

Общая логика:

1. Открыть панель Secrets в Colab.
2. Создать секрет, например `OPENAI_API_KEY`.
3. Включить доступ к секрету для конкретного ноутбука.
4. В коде получить значение секрета.

Пример кода:

```
from google.colab import userdata

api_key = userdata.get("OPENAI_API_KEY")
```

После этого переменную `api_key` можно использовать в запросах к API.

Правильно:

```
from google.colab import userdata

api_key = userdata.get("OPENAI_API_KEY")

if not api_key:
    raise ValueError("Секрет OPENAI_API_KEY не найден в Colab Secrets")
```

Неправильно:

```
api_key = "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

Плюсы:

- удобно для Colab;
- секрет не записан в коде;
- можно управлять доступом к секрету;
- ноутбук можно отправить без самого ключа.

Минусы:

- работает именно в Colab;
- при переносе ноутбука в локальный Jupyter код нужно адаптировать;
- нужно включить доступ секрета к конкретному ноутбуку;
- нельзя печатать секрет на экран.

Когда использовать:

- если занятие проходит в Google Colab;
- если слушатели работают в браузере;
- если нужно подключиться к API из Colab;
- если ноутбук будут пересылать или показывать.

8. Способ 5. Использовать системное хранилище ключей через keyring

Для локальной работы можно использовать системное хранилище секретов: Keychain на macOS, Credential Manager на Windows, Secret Service на Linux. В Python для этого часто применяют библиотеку `keyring`.

Пример установки:

```
pip install keyring
```

Сохранение секрета:

```
import keyring

keyring.set_password("analytics_course", "OPENAI_API_KEY", "ваш_ключ")
```

Получение секрета:

```
import keyring

api_key = keyring.get_password("analytics_course", "OPENAI_API_KEY")
```

Плюсы:

- секрет не лежит в ноутбуке;
- секрет не лежит в `.env` как обычный текстовый файл;
- используется системное хранилище;
- удобно для персональной локальной работы.

Минусы:

- у разных операционных систем разная настройка;
- в учебной группе может быть сложнее поддерживать;
- не всегда удобно в Colab;
- требует установки и объяснения дополнительной библиотеки.

Когда использовать:

- для персональной локальной разработки;
- если ключ нужен часто;
- если не хочется хранить `.env` с секретами;
- если у слушателя уже настроена локальная среда.

9. Способ 6. Использовать менеджеры секретов в облаке

Для производственных задач лучше использовать специализированные менеджеры секретов.

Примеры:

- Google Cloud Secret Manager;
- AWS Secrets Manager;
- Azure Key Vault;

- HashiCorp Vault;
- корпоративные секрет-хранилища.

Идея такая: секрет хранится в защищённом сервисе, а код получает его только во время выполнения и только при наличии прав.

Плюсы:

- централизованное хранение;
- контроль доступа;
- аудит обращений;
- возможность ротации секретов;
- лучше подходит для командной работы;
- подходит для production-процессов.

Минусы:

- требуется настройка облака или корпоративной инфраструктуры;
- нужно понимать роли и права доступа;
- для начального учебного занятия может быть слишком сложно;
- зависит от выбранной платформы.

Когда использовать:

- в рабочих проектах;
- для командной аналитики;
- для production-ноутбуков;
- для регулярных процессов;
- когда секреты дают доступ к реальным данным или платным сервисам.

10. Способ 7. Использовать секреты в GitHub Actions и CI/CD

Если ноутбук или скрипт запускается автоматически через GitHub Actions, токены нельзя хранить в репозитории.

Для этого используют GitHub Secrets.

Общая логика:

1. В настройках репозитория создать secret.
2. Назвать его, например `OPENAI_API_KEY`.
3. Использовать его в workflow как переменную окружения.
4. Не печатать значение секрета в лог.

Пример фрагмента workflow:

```
env:  
  OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
```

В Python-коде:

```
import os

api_key = os.getenv("OPENAI_API_KEY")
```

Плюсы:

- секрет не хранится в репозитории;
- удобно для автоматического запуска;
- можно разделять секреты по репозиториям и окружениям;
- подходит для CI/CD.

Минусы:

- нужно понимать GitHub Actions;
- секрет может случайно попасть в лог, если его напечатать;
- права к репозиторию должны быть настроены аккуратно.

Когда использовать:

- для автоматических проверок;
- для запуска скриптов по расписанию;
- для сборки отчётов;
- для CI/CD;
- для командных проектов на GitHub.

11. Что обязательно добавить в `.gitignore`

Если проект использует токены, нужно добавить в `.gitignore` файлы, которые не должны попадать в репозиторий.

Пример:

```
# Environment files
.env
*.env

# Credentials
credentials.json
service_account.json
kaggle.json
token.json

# Local config
config.local.json
secrets.json
```

```
# Jupyter checkpoints
.ipynb_checkpoints/

# OS files
.DS_Store
Thumbs.db
```

Важно:

`.gitignore` работает только для новых файлов, которые ещё не были добавлены в Git.

Если файл с секретом уже попал в репозиторий, простое добавление в `.gitignore` не удалит его из истории.

12. Как очищать notebook перед отправкой

Перед отправкой ноутбука преподавателю, в LMS или в GitHub нужно выполнить проверку.

Минимальный чек-лист:

1. В коде нет настоящих токенов.
2. В Markdown нет токенов.
3. В выводах ячеек нет токенов.
4. Ошибки не содержат токены.
5. `.env` не отправляется.
6. `credentials.json` не отправляется.
7. `kaggle.json` не отправляется.
8. Ноутбук запускается без хардкода секретов.
9. В README указано, как пользователь должен добавить свой ключ.
10. Все демонстрационные значения заменены на заглушки.

Очистить выводы можно через интерфейс Jupyter:

Kernel / Restart & Clear Output

или через командную строку:

```
jupyter nbconvert --ClearOutputPreprocessor.enabled=True --inplace
notebook.ipynb
```

Можно также использовать инструменты, которые автоматически удаляют выводы из ноутбуков перед коммитом, например `nbstripout`.

Пример:

```
pip install nbstripout
nbstripout --install
```

После этого выводы notebook будут очищаться перед попаданием в Git.

13. Как проверять проект на утечки секретов

Перед публикацией проекта полезно выполнить проверку.

Что искать вручную:

- `api_key` ;
- `token` ;
- `password` ;
- `secret` ;
- `Authorization` ;
- `Bearer` ;
- `PRIVATE KEY` ;
- `DATABASE_URL` ;
- `credentials` ;
- `kaggle.json` ;
- `.env` .

Примеры команд для ручного поиска:

```
grep -R "api_key" .
grep -R "token" .
grep -R "password" .
grep -R "Authorization" .
```

Для более серьёзной проверки используют специальные инструменты:

- GitHub secret scanning;
- GitHub push protection;
- TruffleHog;
- Gitleaks;
- detect-secrets;
- pre-commit hooks.

Главная идея:

проверять нужно не только текущие файлы, но и историю Git.

Если токен был в старом коммите, он может оставаться доступным даже после удаления из последней версии файла.

14. Что делать, если токен уже утёк

Если токен случайно попал в ноутбук, GitHub, LMS, чат или демонстрационный файл, действовать нужно быстро.

Порядок действий:

1. Сразу перестать использовать этот токен.
2. Зайти в сервис, которому принадлежит токен.
3. Отозвать токен или перевыпустить новый.
4. Проверить, не было ли подозрительной активности.
5. Удалить токен из ноутбука и файлов.
6. Очистить выводы notebook.
7. Проверить `.env`, `credentials.json`, `kaggle.json` и другие файлы.
8. Если токен попал в Git, удалить его из истории репозитория.
9. Сообщить преподавателю, администратору или владельцу системы, если токен давал доступ к реальным данным.
10. В новом токене ограничить права и срок действия, если сервис это позволяет.

Важно:

удалить строку из notebook недостаточно.

Если секрет был опубликован, его нужно считать раскрытым и заменить.

15. Принцип минимальных прав

Один из лучших способов снизить риск — выдавать токenu только те права, которые действительно нужны.

Плохой вариант:

- один токен имеет полный доступ ко всем данным;
- токен бессрочный;
- токен используется всеми участниками;
- токен хранится в ноутбуке.

Хороший вариант:

- токен имеет только нужные права;
- токен ограничен по сроку;
- у каждого пользователя свой токен;
- токен можно быстро отозвать;
- токен не хранится в коде;
- для учебных целей используется тестовый или демонстрационный доступ.

Для учебных ноутбуков особенно полезно использовать:

- тестовые API-ключи;

- временные токены;
- ограниченные лимиты;
- синтетические данные;
- обезличенные данные;
- отдельные учебные аккаунты.

16. Как писать безопасные примеры для слушателей

Плохой учебный пример:

```
API_KEY = "sk-real-token-here"
```

Лучше:

```
import os

API_KEY = os.getenv("OPENAI_API_KEY")

if not API_KEY:
    raise ValueError("Добавьте OPENAI_API_KEY в переменные окружения")
```

Для Colab:

```
from google.colab import userdata

API_KEY = userdata.get("OPENAI_API_KEY")

if not API_KEY:
    raise ValueError("Добавьте OPENAI_API_KEY в Colab Secrets")
```

Для разовой демонстрации:

```
from getpass import getpass

API_KEY = getpass("Введите API-ключ: ")
```

Для `.env`:

```
from dotenv import load_dotenv
import os

load_dotenv()
```

```
API_KEY = os.getenv("OPENAI_API_KEY")
```

В README лучше писать так:

Для запуска создайте переменную окружения OPENAI_API_KEY.

Не вставляйте реальный API-ключ в notebook.
Не отправляйте .env-файл в GitHub или LMS.

17. Рекомендуемый вариант для занятий

Для учебных занятий со слушателями можно использовать три уровня.

Уровень 1. Самый простой

Для разовой демонстрации:

```
from getpass import getpass  
  
api_key = getpass("Введите API-ключ: ")
```

Подходит, если слушатели только знакомятся с API и не готовы настраивать окружение.

Уровень 2. Для Google Colab

Для Colab:

```
from google.colab import userdata  
  
api_key = userdata.get("OPENAI_API_KEY")
```

Подходит, если занятие проходит в браузере и ноутбук будет распространяться между слушателями.

Уровень 3. Для локального проекта

Для локального Jupyter:

```
from dotenv import load_dotenv  
import os
```

```
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

И обязательно:

```
.env
*.env
```

в `.gitignore`.

18. Чек-лист перед отправкой notebook

Перед отправкой ноутбука ответьте на вопросы:

- В коде нет реального API-ключа?
- В Markdown нет реального API-ключа?
- В выводах ячеек нет токена?
- Ошибки выполнения не содержат токен?
- `.env` не прикреплен к работе?
- `credentials.json` не прикреплен к работе?
- `kaggle.json` не прикреплен к работе?
- В `.gitignore` добавлены файлы секретов?
- Notebook очищен от выводов?
- В README написано, как добавить свой ключ?
- Используются переменные окружения, Colab Secrets или `getpass`?
- Если токен был опубликован, он уже отозван?

Если хотя бы на один вопрос ответ «нет», notebook нужно доработать перед отправкой.

19. Итоговая памятка

Никогда не вставляйте реальные токены прямо в notebook.

Используйте безопасные варианты:

Ситуация	Подходящий способ
Разовая демонстрация	<code>getpass</code>
Google Colab	Colab Secrets
Локальный Jupyter	переменные окружения или <code>.env</code>
Персональная локальная работа	<code>keyring</code>
Командная production-работа	облачный менеджер секретов
GitHub Actions	GitHub Secrets

Ситуация	Подходящий способ
Публикация notebook	очистка outputs и secret scanning

Главное правило:

секрет должен находиться вне ноутбука, а ноутбук должен только получать его во время выполнения.

Такой подход делает работу безопаснее, помогает избежать случайных утечек и формирует профессиональную привычку: данные, доступы и ключи нужно защищать так же внимательно, как и сам аналитический результат.